

# SALEH YAHYA

714-673-5706 | salehyahya10.20@gmail.com | [salehyahyaa.com](https://salehyahyaa.com) | [linkedin.com/in/salehyahyaa](https://linkedin.com/in/salehyahyaa) | [github.com/salehyahyaa](https://github.com/salehyahyaa)

## EDUCATION

### University of Illinois Urbana-Champaign

Expected Dec 2027

M.S. in Computer Science

Relevant Courses: Parallel Programming, Distributed Systems, Scientific Visualization, Numerical Methods, Data Cleaning, etc

### Lewis University

Expected Dec 2026

B.S. in Computer Science

B.S. in Information Technology

## PROFESSIONAL EXPERIENCE

### Lewis University | Teaching Assistant

Aug 2025 – Dec 2025

- Elected from 20+ students to assist for Object-Oriented Programming class, providing tutoring and coaching with Python & C++.
- Organized 2 weekly study sessions, for collaborative problem solving and stronger understanding of programming principles.
- Improved my students' performance, resulting in a 21% higher average grade compared to the previous semester.

### Paidwork | Machine Learning Engineer Intern

Sep 2025 – Nov 2025

- Improved biometric verification reliability by implementing liveness detection using camera sensors with Python & PyTorch.
- Increased document verification accuracy by refactoring object detection and optical character recognition models across multiple international identification formats, improving text extraction and layout detection by 8% using TensorFlow.
- Strengthened fraud prevention by developing anti spoofing computer vision perception models using depth estimation and texture based analysis, reducing fraudulent authentication from static image and replay attacks by 37% to a user base of 25M.

### OveoAI | Software Engineer Intern

May 2025 – Aug 2025

- Migrated and transformed data pipelines from MySQL to PostgreSQL using Python, increasing database performance by 18%.
- Tuned denial model that automatically classifies denial reasons and generates corrected resubmission suggestions.
- Improved claim denial model accuracy by 12% by incorporating payer rules and coding compliance checks.

## RESEARCH

### MPMC Queue Performance Based Benchmarking | C++ 20, Python, SQL | [Publication](#) | [GitHub](#)

- Benchmarked MPMC queue performance by comparing mutex synchronization against lock-free atomic implementations in C++.
- Designed and executed multi-threaded simulations across 200K–6M operations measuring throughput, latency benchmarking.
- Identified system bottlenecks including memory safety, performance degradation, and lower throughput as data and threads grew.
- Transformed data into database with Python & SQL, visualized data with dashboards, enhanced research efficiency by 20%.

## PROJECTS

### Lum – Financial Data Network & Analytics Platform | Python, Scikit-learn, PostgreSQL, AWS, Redis, Nginx | [luminvy.com](https://luminvy.com) | [GitHub](#)

- Deployed fintech distributed system with AWS EC2, CloudWatch and CI/CD automation for production monitoring and scaling.
- Built asynchronous institutional ingestion workflows in Python, reducing financial management processing time by 40%.
- Reduced REST API response latency by 30% through Redis caching and optimized PostgreSQL indexing strategies.
- Developed ML financial analytics model using Scikit-learn for transaction classification & time series forecasting to 1k+ users.

### Limit Orderbook Matching Engine | C++, NoSQL, GTest | [GitHub](#)

- Implemented a price, time priority limit order book in C++ processing microsecond level order matching for bid and ask levels.
- Designed custom priority queues and event driven logic supporting limit, market, and cancel orders enforcing FIFO execution.
- Simulated exchange style order flow and liquidity dynamics across thousands of order events under varying market conditions.

### Raycast Inspired AI Overlay | Python, Gemini API | [GitHub](#)

- Built GUI AI overlay triggered by native hotkeys for instant LLM access in Python eliminating browser tab-switching disruptions.
- Designed multi-threaded architecture separating input listener, UI thread, and worker queue for thread safe execution.
- Configured HTTP streaming to reduce perceived latency by 40% and cut 3+ manual steps per query by streamlining workflows.

## SKILLS

**Languages:** Python, C/C++, Java, JavaScript, SQL

**Frameworks & Tools:** Kafka, Redis, WebSocket, REST API, PyTest, GTest, Matplotlib

**Infrastructure:** AWS, UNIX/Linux, CUDA, PostgreSQL, NoSQL, GCC, Nginx, Jenkins, Docker, Git, GitHub Actions

**Machine Learning:** PyTorch, TensorFlow, Scikit-learn, Numpy, Pandas